Department of CSE (IoT & Cybersecurity including Blockchain)
B.M.S. College of Engineering, Bengaluru

AROS

Adaptive Resilience & Operational Security For WindowsOS Based Systems

PHASE - 1 Runtime Behavioral Drift Detection Framework

23IC6PWPP1 | Major Project Phase-1 | 6th Semester | AY 2025–26

S Rohith
1BM23IC057

O V Yogeswar Reddy
1BM23IC045

Dadapir N F
1BM24IC400

Project Guide: Prof. Susmitha Mukund Kirsur, Assistant Professor, Dept. of CSE (ICB), BMSCE.

05 JUNE 2026

Table of Contents

01 **Motivation & Problem Statement**

Runtime security gap & motivating threat class

02 **Proposed System**

AROS core concept, scope, and innovation differentiator

03 **Objectives**

O1–O4: Measurable Phase-1 research targets

04 **High-Level Design**

5-layer architecture, system flow, tools, evaluation

05 **Gaps in Existing System**

G1–G6: Six identified research & tooling gaps (2025–26)

01 Motivation

600%

Rise in ransomware attacks (2019–2026)

Source: Cybersecurity Ventures, "Cybercrime Statistics 2025–2026"

287

Average days to detect a breach

Source: IBM Security, "Cost of a Data Breach Report 2025"

$4.9M/breach

Average cost of a data breach (2025)

Source: IBM Security, "Cost of a Data Breach Report 2025"

- Most IDS/IPS systems rely on static signatures ineffective against zero-day & APT attacks.
- Incident response remains largely manual, causing dangerous delays in threat containment.
- No existing production solution integrates detection + RL decision-making + MTD + self-healing in one closed loop.
- Therefore there is an need for an autonomous, adaptive cybersecurity framework that acts without human intervention.

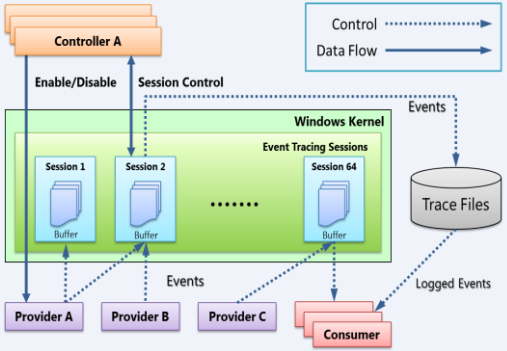
01 Problem Statement

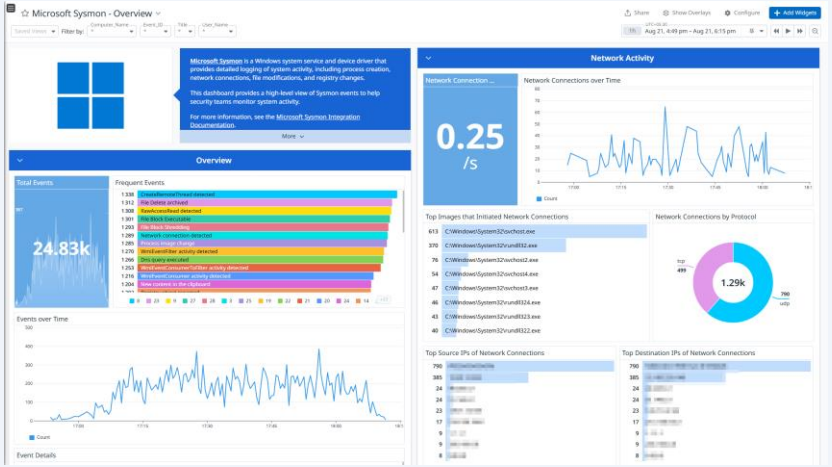
Traditional Windows security systems mainly focus on signature-based detection and short-term anomaly monitoring. However, modern cyber threats such as stealthy malware, insider modifications, configuration drift, persistence mechanisms, and zero-day behavioral changes often evolve gradually during runtime and remain undetected by conventional security tools. Existing solutions generate large volumes of alerts but lack adaptive behavioral understanding of long-term system operations.

Windows environments provide rich runtime telemetry sources such as Event Tracing for Windows (ETW), Sysmon, Event Logs, Performance Counters, and WMI (Windows Management , yet these data sources are not effectively utilized for continuous behavioral drift analysis. As a result, organizations face difficulty in identifying silent operational deviations before they become critical security incidents.

Therefore, there is a need for an adaptive runtime security framework capable of continuously monitoring Windows-based systems, establishing normal behavioral baselines, detecting long-term behavioral drift, and improving operational resilience against evolving and previously unseen threats with minimal system overhead.

ETW Architecture





02 Proposed System AROS

Core Concept: Behavioral Identity

AROS treats each monitored service as having a behavioral identity a stable, multi-dimensional statistical profile derived from runtime telemetry under normal operating conditions. Persistent, statistically significant drift from this identity triggers a security-relevant flag.

What AROS Does (Phase 1)

- Collects CPU, memory, process, network & timing telemetry from a target Windows service
- Constructs a multi-dimensional behavioral baseline during defined normal operation
- Applies statistical distribution comparison & lightweight ML to detect persistent drift
- Evaluates performance: detection latency, FPR, accuracy, overhead
- Generates structured logs & drift score visualizations

Scope Boundaries

- Phase 1 deliverables: Detection · Analysis · Baseline Modeling · Evaluation · Literature Survey · Design
- Platform: Windows 10/11 (64-bit) local controlled environment
- Recovery & self-healing: architecturally specified but NOT implemented in Phase 1
- Formally scoped as Phase-2 extension to avoid overclaiming
- No real-world network access fully sandboxed laboratory setting

Innovation Differentiator

- Long-term behavioral drift not immediate anomaly detection
- Models the identity of an individual service not generic network rules
- Complete measurable pipeline: telemetry → drift quantification → evaluation
- Results are reproducible and academically defensible
- Formalized for Windows runtime an under-addressed research domain

03 Objectives — Phase 1

Four measurable research & development targets — specific, technically defensible, and evaluable within the project timeline.

O1

Instrument & Collect Runtime Telemetry

Design & implement a lightweight telemetry collection module for Windows capturing CPU usage, memory patterns, process activity metrics, network behavior, and system call timing from a target service at configurable sampling intervals.

O2

Construct a Behavioral Baseline Model

Develop a baseline modeling component that aggregates telemetry over a defined normal-operation window and produces a stable, multi-dimensional statistical profile representing behavioral identity — accommodating workload-driven variation without false drift signals.

O3

Detect Persistent Behavioral Drift

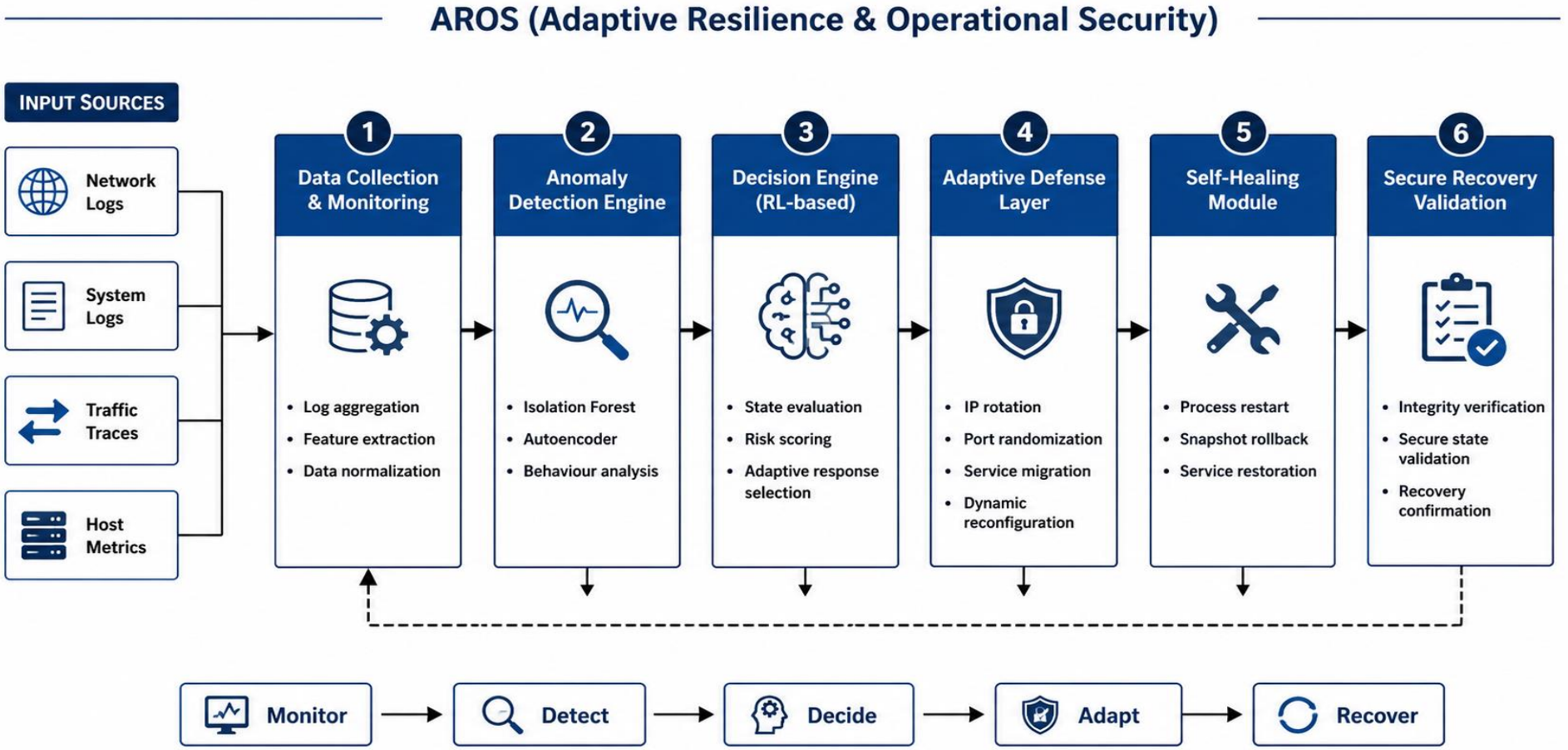
Implement a drift detection engine applying statistical distribution comparison and/or lightweight unsupervised ML to identify persistent divergence from the established baseline, with drift severity expressed as a quantifiable score.

O4

Evaluate the Framework Against Defined Metrics

Assess detection performance through controlled experiments measuring: (a) Detection Latency, (b) False Positive Rate (FPR), (c) Monitoring Overhead (CPU/memory), and (d) Drift Sensitivity across gradual-change scenarios.

04 High-Level Design — Architecture Layers



04.1 High-Level Design — Architecture Layers

Modular 5-layer sequential pipeline — each layer independently testable before integration.

Layer	Component	Inputs	Processing	Output
1	**Telemetry Collection**	Windows service runtime signals	Periodic metric sampling; normalization	Timestamped feature vectors
2	**Pre-processing & Feature Eng.**	Raw metric time series	Windowing; aggregation; outlier filtering	Cleaned, windowed dataset
3	**Baseline Modeling**	Normal-operation dataset	Statistical profile construction; variance learning	Behavioral identity model
4	**Drift Detection Engine**	Live feature vectors + baseline	Distribution comparison; drift scoring; persistence check	Drift score; alert flag
5	**Analysis & Evaluation**	Drift scores; alert logs	Metric computation (latency, FPR, overhead); visualization	Evaluation report; drift timeline charts

04 System Flow & Processing Pipeline

Layer 1



Windows Runtime Telemetry

CPU · Memory · Process · Network · Timing

Layer 2



Pre-processing & Feature Engineering

Windowing · Aggregation · Outlier Filtering

Layer 3



Baseline Modeling

Mean · Variance · Distributional Profile

Layer 4



Drift Detection Engine

Distribution Comparison · Drift Scoring · Persistence

Layer 5



Alert & Analysis Output

Metrics · Evaluation Report · Timeline Charts

↑ *Feedback Loop: Confirmed normal-behavior windows incrementally update the behavioral baseline to accommodate legitimate long-term evolution.*

Drift Event Criteria: Score exceeds defined tolerance threshold AND persists across multiple consecutive windows → Event logged with timestamp, affected metrics & drift magnitude

04 Tools & Platform

Component	Tool / Technology	Purpose
Host Platform	Windows 10 / 11 (64-bit)	Target runtime environment
Telemetry Collection	Python 3.x + psutil / WMI / ETW / Sysmon	Metric sampling & event stream extraction
Event Logging	Sysmon + Windows EVTX (Event Log)	Process, network & registry event capture
Data Storage	SQLite / CSV	Baseline and runtime data persistence
ML / Statistics	scikit-learn, NumPy, SciPy	Drift detection modeling
Visualization	Matplotlib / Streamlit	Drift score charts and dashboards
Attack Simulation	Controlled config changes / Kali VM	Generating controlled drift scenarios
Deployment Mode	Agent-Based (SYSTEM) or Agentless (WMI/WinRM)	Flexible on-host or remote collection
Version Control	Git / GitHub	Source management and reproducibility
Development IDE	VS Code	Code development and debugging

Evaluation Metrics

Detection Latency	Drift intro → alert time
FPR	Incorrect alerts under normal load
Drift Accuracy	Correctly identified events
Overhead	AROS CPU & memory usage
Score Stability	Variance during normal op

04 Evaluation Framework

Controlled Experiment Setup

- Deploy target Windows service in isolated lab environment
- Monitor service with AROS telemetry collection module
- Run an initial observation phase to establish behavioral baseline
- Introduce controlled drift scenarios at known time points:
  → Configuration changes
  → Process lifecycle modifications
  → Gradual behavioral shifts (simulated via Kali VM)
- Compare detection outcomes against ground truth labels
- Compute all four evaluation metrics from accumulated alert log
- Generate drift score visualizations and evaluation report

M1

Detection Latency

Time elapsed from drift introduction to first drift alert generation. Lower = better responsiveness.

M2

False Positive Rate (FPR)

Proportion of normal-behavior windows incorrectly flagged as drift. Target: < 5% under standard workload.

M3

Drift Detection Accuracy

Correctly identified drift events as a fraction of all introduced drift scenarios. Target: > 90%.

M4

Monitoring Overhead

Additional CPU and memory utilization attributable to AROS telemetry collection and processing pipeline.

M5

Drift Score Stability

Variance of drift scores during confirmed normal operation periods. Low variance indicates reliable baseline.

05 Literature Survey

01 Research Problem Scoping

Identified core gap: existing IDS miss long-term behavioural drift in Windows services.

02 Keyword Formulation

Terms: IDS, anomaly detection, ETW, behavioural modelling, zero-day, Windows telemetry.

03 Database Search

Queried IEEE Xplore, Google Scholar, ACM DL, ScienceDirect — 2020–2025 range.

04 Initial Screening

40+ papers reviewed; filtered by relevance, recency, and citation count.

05 Deep Reading & Annotation

Continues ↓

7 core papers studied in depth — methodology, datasets, outcomes, and limitations.

06 Methodology Comparison

ML/DL vs RL vs hybrid approaches — evaluated for adaptability, overhead, and generalisability.

07 Gap Analysis

Three critical gaps: real-time bias, no behavioural identity modelling, no drift persistence.

08 Reference Shortlisting

7 papers finalised as primary references; mapped to AROS objectives and design choices.

09 Project Direction Shaped

AROS framework designed: drift-based detection, ETW/Sysmon pipeline, agent model.

05 Gaps in Existing System (2025–26)

Six critical research and tooling gaps identified through literature review:

G1

Real-Time Bias in Detection Systems

Existing IDS and anomaly detection systems analyze discrete events or short time windows. They are designed for sudden attacks but cannot detect threats that evolve over days or weeks. Long-term behavioral drift falls entirely outside their detection horizon.

G2

Absence of Behavioral Identity Modeling

No current tool models the 'behavioral identity' of a specific software service. Generic thresholds and network-layer signatures cannot distinguish between legitimate workload-driven variation and security-relevant drift.

G3

Lack of Drift Persistence Analysis

Even statistical anomaly tools evaluate each time window in isolation — they cannot determine if a deviation is transient (e.g., a maintenance task) or persistent (e.g., a backdoor gradually increasing outbound traffic). This causes high false-positive rates and missed chronic threats.



Root Cause

All three gaps stem from the same fundamental design decision: existing tools optimize for immediate precision, not temporal persistence. AROS addresses this by tracking behavioral identity over long time horizons.

05 Gaps in Existing System (2025–26)

Continued — structural gaps specific to Windows-based service-level monitoring:

G4

Windows Runtime Security Under-Addressed

Most research on runtime monitoring, behavioral modeling, and drift detection targets Linux or cloud-native environments. Windows-based systems — dominant in enterprise and educational settings — receive comparatively little behavior-centric security research attention.

G5

No Integrated Detect-Analyze-Evaluate Pipeline

Existing research isolates single aspects — feature engineering, ML model accuracy, or system overhead. There is no integrated pipeline from telemetry collection through drift quantification with measurable outcomes in a controlled Windows environment.

G6

Recovery Without Detection Foundation

Self-healing and adaptive recovery are active research areas, but the foundational problem of accurate behavioral drift detection — a prerequisite for meaningful recovery — remains underexplored for Windows runtime service-level monitoring.

Gap Summary Matrix

Gap	Domain	Impact
G1–G3	Detection Design	High FPR / Missed drift
G4	Platform Coverage	Windows gap in research
G5	Pipeline Integration	No end-to-end eval
G6	Recovery Readiness	No detection foundation

05. Literature Survey (1/2)

P.No	Title of Paper	Year	Key Outcome	Gap Identified
1	Machine Learning and Deep Learning Methods for Cybersecurity: A Survey	2025	Reviews ML/DL techniques for IDS, including modern datasets, adversarial attacks, and AI-based detection.	Current IDS lack adaptability, real-time response, and robustness against modern cyber threats.
2	AI-Driven Cybersecurity Framework for Anomaly Detection	2025	LSTM + Random Forest anomaly detection with high accuracy.	No RL decision-making, dynamic reconfiguration, or autonomous recovery.
3	Autonomous Cyber-Physical Security Middleware for IoT	2025	Multi-layer architecture with anomaly detection and automated mitigation.	Limited RL integration; no complete closed-loop adaptive security framework.

Literature Survey (2/2)

P.No	Title of Paper	Year	Key Outcome	Gap Identified
4	Artificial Intelligence in Cybersecurity: Emerging Trends	2025	AI applications in intrusion detection, malware analysis, and autonomous security.	Theoretical only; no implementation of an integrated adaptive framework.
5	A Systematic Survey on Intrusion Detection using Deep Learning in Cybersecurity	2025	Covers deep learning architectures (CNN, RNN, transformers) for intrusion detection.	Traditional ML-based IDS assume fixed environments, lack adaptability to evolving threats, and fail in real-world dynamic network conditions.
6	Adaptive Learning for Moving Target defence: Enhancing Cybersecurity Strategies	2025	Reinforcement learning + game theory to dynamically adapt defense strategies against attackers. Improves system security by enabling adaptive, evolving defense mechanisms against changing threats	Limited real-world validation, Focus mainly on simulation-based testing
7	Reinforcement Learning for Autonomous Resilient Cyber Defence	2025	Reinforcement learning for autonomous cyber defence decision making, Enables self-adaptive and resilient response to dynamic cyber threats	Reward design difficulty, Incomplete system visibility, Poor generalization.

06 REVIEW OF PROJECT

TP1 What problem are you solving?

Most security tools detect immediate attacks or known signatures. AROS addresses a different threat class: slow, stealthy behavioral drift in runtime systems that develops over days or weeks and is missed by conventional alerting. We model what normal looks like for a specific Windows service and detect when that changes persistently.

TP2 Why is this novel not just another IDS?

An IDS looks for sudden deviations or known signatures. AROS looks for gradual departure from a service-specific behavioral identity over long time windows. The novelty lies in modeling behavioral identity, applying drift persistence checks, and producing quantified drift scores not matching rules or signatures.

TP3 How will you measure success?

We define four concrete metrics: detection latency, false positive rate, drift detection accuracy, and monitoring overhead. All are computed from controlled experiments where drift is introduced at known time points — making results reproducible and comparable against ground truth.

TP4 What is the scope and why is recovery not in Phase 1?

Phase 1 delivers the complete detection and analysis pipeline with measurable evaluation. Recovery is architecturally designed but not implemented in Phase 1. This is intentional — recovery without an accurate detection foundation would produce unstable results. We avoid overclaiming.



AROS Research Positioning: Fills the gap between network-level IDS research and cloud self-healing providing service-level behavioral identity modeling for Windows runtime environments. A research prototype with formalized, measurable outcomes.

06 Guide Review Talking Points

TP5	**Is this realistic for a student project?**
	Yes. We monitor one Windows service in a controlled local environment using standard Python libraries and system telemetry. No heavy compute, no GPU, no real-world network access. The scope is bounded and executable within the Phase-1 timeline. The controlled lab setting ensures reproducibility.
TP6	**What is the research contribution?**
	The formalization of behavioral drift detection for Windows runtime systems: defining what constitutes behavioral identity, designing persistence-based drift scoring, and evaluating the pipeline with quantified metrics. This positions AROS as a research prototype rather than a product implementation.